

El "Segundo Cerebro" Semántico (Knowledge Graph)

Este proyecto apunta al dominio de "Productividad y Utilidad", pero lo eleva mediante estructuras de datos avanzadas e integración de IA local. El concepto es un sistema de **Gestión de Conocimiento Personal (PKM)** centrado en la privacidad.

A diferencia de una aplicación de notas estándar, esta aplicación utiliza **Embeddings Vectoriales** para crear enlaces semánticos entre notas automáticamente. Ejecuta un modelo de *embedding* localmente en el dispositivo, convierte las notas del usuario en vectores matemáticos, los almacena y permite una funcionalidad de "Chat con tus Notas" o "Búsqueda Semántica" — todo sin que un solo byte abandone el dispositivo.

Palabras Clave para el CV: SwiftData, CoreML, NaturalLanguage, Vector Search, Local-First, Cryptography, Modular Architecture, Accelerate.

3.1 Diseño Arquitectónico y Modularización

La aplicación debe arquitecturarse como un conjunto de paquetes Swift débilmente acoplados:

1. **CoreDataLayer:** Encapsula la pila de SwiftData, la configuración de sincronización de CloudKit y la lógica de encriptación.
2. **InferenceEngine:** Un módulo puramente funcional que envuelve CoreML. Toma cadenas de texto y devuelve arrays de vectores (`[Float]`). Este módulo debe ser agnóstico de la UI.
3. **AppFeature Modules:** Módulos distintos para "Editor", "Búsqueda" y "Visualización de Gráfico".

3.1.1 La Capa de Datos: Gráfico vs. Relacional

Las aplicaciones de notas estándar utilizan una estructura de carpetas jerárquica. Un "Segundo Cerebro" utiliza un gráfico.

- **Modelo de Datos:** Una entidad `Note` tiene una relación de muchos a muchos con `Tag` y una relación reflexiva con otras `Note` (enlaces).
- **El Desafío:** CloudKit tiene límites en la complejidad de las relaciones y referencias. Sincronizar miles de relaciones pequeñas puede saturar el motor de sincronización.
- **La Solución Senior:** En lugar de depender únicamente de las relaciones de Core Data, implementar un sistema de "enlaces blandos" (*soft links*) utilizando UUIDs en el cuerpo del texto, analizados en tiempo de ejecución.

Esto previene la "saturación de relaciones" durante la sincronización inicial y demuestra un conocimiento de las limitaciones de CloudKit.

3.2 Implementación de Búsqueda Vectorial Local

Este es el factor diferenciador técnico ("Wow factor").

- **Selección del Modelo:** Utilizar un modelo BERT-SQuAD cuantizado o un modelo de *embedding* moderno (como `all-MiniLM-L6-v2`) convertido a CoreML. El modelo debe pesar menos de 50MB para ser viable en dispositivos móviles.
- **Flujo de Proceso:**
 1. El usuario escribe una nota.
 2. Se activa una `BackgroundTask` (utilizando el modificador `.backgroundTask` de SwiftUI) para no bloquear la escritura.
 3. El texto se pasa al `InferenceEngine`.
 4. El vector resultante (ej. 384 dimensiones) se almacena. Dado que SwiftData no está optimizado para búsquedas vectoriales nativas de alto rendimiento, el vector se almacena como un blob binario (`Data`).
 5. **Búsqueda:** Cuando el usuario consulta "ideas para vacaciones", la consulta se vectoriza y se ejecuta una búsqueda de **Similitud de Coseno** contra los vectores de las notas almacenadas.
- **Optimización de Bajo Nivel:** Ejecutar Similitud de Coseno en 10,000 vectores en Swift puro puede ser lento. Un desarrollador senior utiliza el framework **Accelerate (vDSP)** de Apple para realizar operaciones matemáticas vectoriales en la CPU/GPU de manera SIMD (*Single Instruction, Multiple Data*). Esto demuestra habilidades de optimización de rendimiento de bajo nivel y conocimiento profundo del hardware.

3.3 Desafíos de Ingeniería y Soluciones

Desafío A: Conflictos de Sincronización en Texto

Escenario: Un usuario edita una nota en un iPad y un iPhone simultáneamente mientras está desconectado. **Enfoque Junior:** "El último que escribe gana" (una edición se pierde). **Enfoque Senior:** Implementar un **Tipo de Dato Replicado Libre de Conflictos (CRDT)** para el cuerpo del texto, o una estrategia de fusión semántica.

- **Implementación Práctica:** Si un CRDT completo es demasiado complejo para el alcance, una solución senior pragmática es detectar la notificación `NSPersistentCloudKitContainerEventChangedNotification`, identificar el

conflicto de versión y presentar una UI de "Resolución de Conflictos" (estilo git merge) al usuario. Esto muestra respeto por la integridad de los datos del usuario.

Desafío B: Privacidad y Encriptación en Reposo

Escenario: El usuario quiere asegurar que Apple no pueda leer sus notas en iCloud. **Solución:** Implementar **Encriptación a Nivel de Aplicación**.

- Antes de que los datos se guarden en SwiftData (y se sincronicen con CloudKit), el contenido se encripta utilizando `CryptoKit` (ChaCha20-Poly1305) con una clave derivada del código de acceso del dispositivo o una clave protegida biométricamente almacenada en el *Secure Enclave*.
- **Valor en el CV:** Esto demuestra conocimiento de seguridad, Keychain y `CryptoKit`, aumentando enormemente el valor del candidato para roles en fintech o health-tech.

3.4 Aspectos Destacados de UI/UX

- **Visualización de Gráfico:** Usar `SwiftUI Canvas` o `Metal` para renderizar un gráfico dirigido por fuerza (*force-directed graph*) de las notas. Esto requiere matemáticas de diseño complejas y ajuste de rendimiento para renderizar más de 500 nodos a 120Hz sin caídas de cuadros.
- **Editor de Texto:** No usar el `TextEditor` estándar. Envolver `UITextView` a través de `UIViewRepresentable` para acceder a `NSLayoutManager` (TextKit 2). Esto permite el resaltado de sintaxis personalizado (markdown) y dibujar "líneas de conexión" personalizadas entre wikilinks dentro del texto.